

Data Structures: Practice Questions

15 short-answer questions

Instructions. Answer each question and briefly justify your answer. Show intermediate steps for algorithm traces and recurrence calculations. For complexity questions, give a tight asymptotic bound using Θ notation.

Assumptions. Arrays have spare capacity for insertion. Memory allocation and key comparisons take constant time. Linked lists are singly linked unless stated otherwise. BSTs are unbalanced and contain distinct keys. Recurrences have $T(1) = \Theta(1)$; ignore rounding of subproblem sizes.

1. **Bubble sort trace.** Sort $[6, 2, 5, 1, 4]$ in ascending order using bubble sort with left-to-right passes. Write the array after each of the first two complete passes, shortening the unsorted prefix after each pass.
2. **Bubble sort complexity.** A bubble-sort implementation stops when a complete pass makes no swaps. Give its best-case and worst-case running times, and describe an input arrangement that achieves each case.
3. **Merging sorted arrays.** Merge $[2, 7, 11]$ and $[3, 5, 9, 12]$ into one sorted array. How many key comparisons does the standard two-pointer merge use on these inputs? Do not count copying elements after one array is exhausted.
4. **Quicksort partition.** An array contains $[8, 3, 10, 1, 6, 4]$. Quicksort chooses 6 as pivot. List the keys belonging to each recursive subproblem after partitioning, and give the pivot's final zero-based index. The order of keys within each subproblem need not be specified.
5. **Quicksort worst case.** Quicksort always selects the last element as pivot. It is applied to an ascending array of n distinct keys using a linear-time partition procedure. Write a recurrence for its running time and give the resulting tight asymptotic bound.

6. **Master Theorem statement.** For $T(n) = aT(n/b) + f(n)$, state the restrictions on the constants a and b . State the three standard cases by comparing $f(n)$ with $n^{\log_b a}$, including the regularity condition needed in the third case.

7. **Master Theorem application.** Solve $T(n) = 4T(n/2) + n^2$ using the Master Theorem. Identify a , b , and $f(n)$, compute $n^{\log_b a}$, and state the applicable case.

8. **Another recurrence.** Use the Master Theorem to solve $T(n) = 3T(n/3) + n^2$. Explicitly verify the regularity condition for this recurrence.

9. **Binary search trace.** Search for 19 in $[2, 5, 9, 13, 19, 24, 31, 40]$. Use inclusive zero-based bounds and $\text{mid} = \lfloor (\text{low} + \text{high})/2 \rfloor$. For every iteration, write the low, high, and midpoint indices and the key examined.

10. **Binary search and linked lists.** State the worst-case running time of binary search on a sorted array. Explain why its usual constant-time midpoint access is unavailable on an ordinary sorted singly linked list with only a head pointer.

11. **Array operations.** For both a sorted array and an unsorted array containing n elements, give the worst-case time for: (i) searching for a supplied key, (ii) inserting an arbitrary new key without checking for duplicates, and (iii) deleting an element at a known index. Preserve sorted order in the sorted array; order need not be preserved in the unsorted array. Briefly explain how each deletion is performed.

12. **Linked-list operations.** For both a sorted and an unsorted singly linked list containing n nodes, with only a head pointer, give the worst-case time for: (i) searching for a supplied key, (ii) inserting an arbitrary new key without checking for duplicates, and (iii) deleting a node by a supplied key, including finding it. Preserve order in the sorted list; choose an efficient insertion position in the unsorted list.

13. **BST construction and search.** Insert 50, 30, 70, 20, 40, 60, 80, 35 into an initially empty BST. Draw the resulting tree, then list the keys visited when searching for 35.

14. **BST successor and predecessor.** Using the BST from Question 13, find the inorder successor of 40 and the inorder predecessor of 60. Explain how each is found in the tree.

15. **BST deletion.** Start again with the original BST from Question 13. Delete 30 using its inorder successor as the replacement. Draw the resulting tree and identify which node is removed from its original position.